

Aula 14: Modelos VEC

Silvio Costa

2026-06-08

Introdução ao VECM

- Quando há cointegração, o modelo VAR original pode ser reescrito na forma de correção de erros, conhecido como VECM (Vector Error Correction Model).
- Vamos tomar como base a seguinte relação de cointegração:

$$u_t = (y_t - z_t) \begin{pmatrix} 1 \\ \beta \end{pmatrix}$$

- Para transformar um modelo VAR em um modelo VEC, precisamos reescrever o modelo VAR de forma a obter o vetor u_t estacionário que representa a relação de longo prazo entre as variáveis.
- Para o caso bivariado $X_t = [y_t, z_t]'$, a introdução do vetor u_t pode ser visualizada no modelo em diferenças como:

$$\Delta y_t = \alpha_1 \hat{u}_{t-1} + \sum_{j=1}^{p-1} \lambda_{11,j+1} \Delta y_{t-j} + \sum_{j=1}^{p-1} \lambda_{12,j+1} \Delta z_{t-j} + e_{yt}$$

$$\Delta z_t = \alpha_2 \hat{u}_{t-1} + \sum_{j=1}^{p-1} \lambda_{21,j+1} \Delta y_{t-j} + \sum_{j=1}^{p-1} \lambda_{22,j+1} \Delta z_{t-j} + e_{zt}$$

- **Importante:** não incluir o vetor $\hat{u}_t$ nesse caso é equivalente a omitir uma variável explicativa, o que pode resultar em estimativas viesadas.
- O modelo VEC é uma versão mais completa e robusta do VAR.

Derivação do modelo VEC

- Note que um modelo VAR em nível pode ser reescrito utilizando operadores de defasagem:

$$X_t = \Phi_1 X_{t-1} + \Phi_2 X_{t-2} + \cdots + \Phi_p X_{t-p} + e_t$$

$$[I - (\Phi_1 L + \Phi_2 L^2 + \cdots + \Phi_p L^p)] X_t = e_t$$

$$\Phi(L) X_t = e_t$$

- $\Phi(L)$ é uma forma funcional apenas para tornar a expressão mais compacta.
- Agora seja o caso particular com $L = 1$:

$$\Phi(1) = [I - (\Phi_1 + \Phi_2 + \dots + \Phi_p)] := -\Phi$$

Exemplo: derivação do VECM a partir de um VAR(2)

- Vamos derivar o modelo VECM partindo de um VAR(2):

$$X_t = \Phi_1 X_{t-1} + \Phi_2 X_{t-2} + e_t$$

- Subtraindo e somando $\Phi_2 X_{t-1}$:

$$X_t = \Phi_1 X_{t-1} + \Phi_2 X_{t-1} - \Phi_2 X_{t-1} + \Phi_2 X_{t-2} + e_t = X_{t-1}(\Phi_1 + \Phi_2) - \Phi_2 \Delta X_{t-1} + e_t$$

- Subtraindo X_{t-1} de ambos os lados:

$$X_t - X_{t-1} = -X_{t-1} + X_{t-1}(\Phi_1 + \Phi_2) - \Phi_2 \Delta X_{t-1} + e_t$$

$$\Delta X_t = -[I - (\Phi_1 + \Phi_2)]X_{t-1} - \Phi_2 \Delta X_{t-1} + e_t$$

$$\Delta X_t = \Phi X_{t-1} - \Phi_2 \Delta X_{t-1} + e_t$$

- Caso haja cointegração, a primeira parte da equação representa o componente de longo prazo.
 - Para entendermos se o sistema está de fato **em equilíbrio**, precisamos avaliar o **comportamento da matriz** Φ , em especial se o vetor de cointegração faz parte dela.
-

Exemplo: derivação do VECM a partir de um VAR(0)

- Neste caso particular, a derivação é ainda mais simples:

$$y_t = \pi x_t + e_t$$

- Subtraindo y_{t-1} de ambos os lados:

$$y_t - y_{t-1} = \pi x_t - y_{t-1} + e_t$$

$$\Delta y_t = \pi x_t - \pi x_{t-1} + \pi x_{t-1} - y_{t-1} + e_t$$

$$\Delta y_t = \pi \Delta x_t - (y_{t-1} - \pi x_{t-1}) + e_t$$

$$\Delta y_t = \pi \Delta x_t - \begin{pmatrix} y_{t-1} & x_{t-1} \end{pmatrix} \begin{pmatrix} 1 \\ -\pi \end{pmatrix} + e_t$$

- Na prática, este modelo é estimado em duas etapas:

1. Estime os resíduos $\hat{u}_t = y_t - \hat{\beta}_1 x_t$;

2. Inclua os resíduos na regressão em diferença:

$$\Delta y_t = \pi \Delta x_t + \alpha \hat{u}_{t-1} + e_t$$

Em que $\hat{u}_{t-1}$ pode ser expresso pelo produto das matrizes $X_{t-1}\beta'$.

Teste de Cointegração de Johansen

- Intuição: Perceba a semelhança entre o teste de Johansen e o teste de raiz unitária Dickey-Fuller Aumentado (ADF).
- Partindo-se do VAR(p) com k variáveis, obtém-se o VAR reparametrizado representado por:

$$\Delta Y_t = \Pi Y_{t-1} + \sum_{i=1}^{p-1} \Gamma_i \Delta Y_{t-i} + \varepsilon_t$$

- O **determinante de uma matriz** é igual ao **produto dos autovalores!!**
- Os pacotes econométricos calculam os autovalores e os ordenam em ordem decrescente.
- O posto r da matriz $n \times n$ pode ser interpretado da seguinte forma:

$$\begin{cases} r < n & \text{há cointegração} \\ r = 0 & \text{não há cointegração, variáveis são não estacionárias} \\ r = n & \text{variáveis endógenas são todas estacionárias} \end{cases}$$

- Lembrando que o **posto da matriz** é o **número de raízes diferentes de zero**.
- Se não há cointegração, todos os autovalores estarão próximos de zero.
- O teste de Johansen fornece três possibilidades com base no posto da matriz Π :

1. Posto(Π) = 0

Isso implica que $\Pi = 0$, o que é análogo ao caso em que $\phi = 0$ no teste ADF. Temos k raízes unitárias. Se não há cointegração, todos os autovalores estarão próximos de zero.

Interpretação: não há relação de cointegração entre as variáveis e não existe mecanismo de correção de erro.

Decisão: o modelo VAR deve ser especificado em primeiras diferenças.

2. Posto(Π) = k

As linhas de Π são linearmente independentes e $|\Pi| \neq 0$. Existem k combinações estacionárias das variáveis, mas a cointegração não é pertinente, pois as variáveis já são estacionárias.

Decisão: o modelo VAR deve ser estimado em nível.

3. $0 \leq \text{Posto}(\Pi) = r \leq k$

Existem r combinações lineares estacionárias e $|\Pi| = 0$, com pelo menos uma linha ou coluna nula. Se as variáveis são $I(1)$, então existem r relações de cointegração que fornecem r vetores de cointegração. O termo ΠY_{t-1} fornece as combinações lineares estacionárias.

Decisão: o modelo VEC (VECM) deve ser utilizado.

- O procedimento de Johansen consiste em testar o número de raízes características diferentes de zero na matriz Π , o que corresponde ao número de relações e vetores de cointegração entre as variáveis.
- São utilizados dois testes:

Teste do Traço

- Este teste considera como hipótese nula a existência de r_0 raízes características diferentes de zero (ou seja, r_0 vetores de cointegração), contra a alternativa de que $r > r_0$. Formalmente:

$$H_0 : r = r_0 \quad \text{vs.} \quad H_1 : r > r_0$$

- O teste analisa a **estatística do traço**, que, no caso de autovalores próximos de zero, apresenta valores muito pequenos. A estatística de teste é dada por:

$$\lambda_{\text{traço}} = -T \sum_{i=r_0+1}^k \ln(1 - \hat{\lambda}_i)$$

Onde: - T é o número de observações; - $\hat{\lambda}_i$ são as raízes características obtidas da matriz Π estimada.

Teste da Raiz Característica Máxima

- Este teste considera como hipótese nula a existência de r_0 raízes características diferentes de zero, contra a alternativa de que $r = r_0 + 1$. Formalmente:

$$H_0 : r = r_0 \quad \text{vs.} \quad H_1 : r = r_0 + 1$$

- A estatística de teste é:

$$\lambda_{\text{máx}} = -T \ln(1 - \hat{\lambda}_{r_0+1})$$

- Os testes são realizados em sequência, de forma crescente, até que a hipótese nula não seja rejeitada.
- Para $H_0 : r = 0$, rejeitar H_0 significa que há um ou mais vetores de cointegração (pelo teste do traço) e exatamente um (pelo teste da raiz máxima).
- Para $H_0 : r = 1$, rejeitar H_0 indica que há dois ou mais vetores de cointegração (traço) e mais um (máxima).

Teorema de Granger

É possível decompor Φ em duas matrizes multiplicativas quando Φ tem posto menor que n (presença de raiz unitária torna o determinante nulo). Pelo teorema de Granger, se $|\Phi| = 0$, podemos escrever:

$$\Phi = \alpha\beta'$$

em que β é o vetor de cointegração.

Logo, o modelo VECM pode ser escrito como:

$$\Delta X_t = (\alpha\beta')X_{t-1} - \Phi_2\Delta X_{t-1} + e_t$$

Exemplo no R: teste de Johansen e modelo VEC

1. Obtendo dados no Google Finance. O pacote quantmod permite baixar dados financeiros.

```

library(tseries)

## Registered S3 method overwritten by 'quantmod':
##   method      from
##   as.zoo.data.frame zoo

library(quantmod)

## Loading required package: xts
## Loading required package: zoo

##
## Attaching package: 'zoo'

## The following objects are masked from 'package:base':
##
##   as.Date, as.Date.numeric

## Loading required package: TTR

# Baixar dados de preços ajustados
getSymbols(c("^SPX", "^IXIC", "^DJI"), src = "yahoo", from = "2001-01-01")

## [1] "SPX" "IXIC" "DJI"

# Selecionar preços ajustados
spx <- Ad(SPX)
nasdaq <- Ad(IXIC)
dji <- Ad(DJI)

# Combinar em um único objeto
data <- na.omit(merge(nasdaq, spx, dji))
colnames(data) <- c("QQQ", "SPX", "DJI")

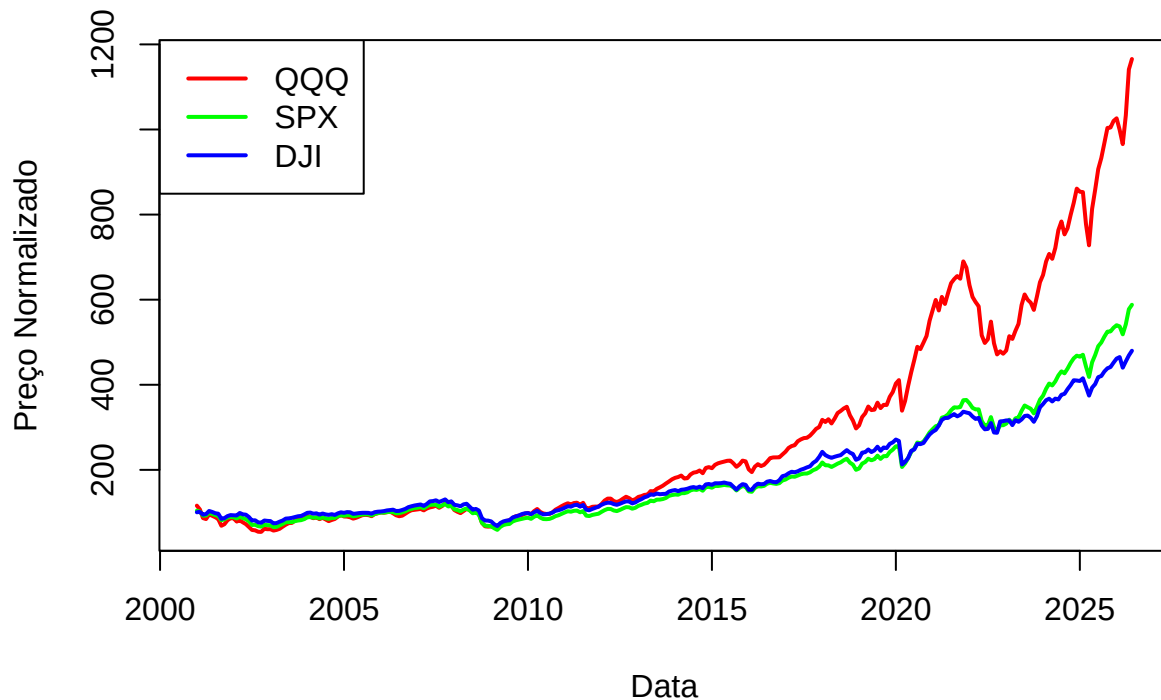
datam <- aggregate(data, as.yearmon, mean)

# Normalizar para 100
norm_prices <- sweep(datam, 2, data[1, ], FUN = "/" ) * 100

# Plotar
matplot(index(norm_prices), coredata(norm_prices), type = "l", lty = 1, lwd = 2,
        col = rainbow(ncol(norm_prices)), xlab = "Data", ylab = "Preço Normalizado",
        main = "Séries Normalizadas (Base 100)")
legend("topleft", legend = colnames(norm_prices), col = rainbow(ncol(norm_prices)), lty = 1, lwd = 2)

```

Séries Normalizadas (Base 100)



2. Testes de raiz unitária. Primeiro, testamos se as séries são integradas de ordem 1 ($I(1)$) com o teste de Dickey-Fuller aumentado (ADF):

```
adf.test(log(datam$QQQ))           # Deve ser não estacionária
```

```
##
## Augmented Dickey-Fuller Test
##
## data: log(datam$QQQ)
## Dickey-Fuller = -2.7319, Lag order = 6, p-value = 0.2678
## alternative hypothesis: stationary
```

```
adf.test(log(datam$SPX))           # Deve ser não estacionária
```

```
##
## Augmented Dickey-Fuller Test
##
## data: log(datam$SPX)
## Dickey-Fuller = -2.2745, Lag order = 6, p-value = 0.4606
## alternative hypothesis: stationary
```

```
adf.test(log(datam$DJI))           # Deve ser não estacionária
```

```
##
## Augmented Dickey-Fuller Test
##
## data: log(datam$DJI)
## Dickey-Fuller = -2.6132, Lag order = 6, p-value = 0.3178
## alternative hypothesis: stationary
```

3. Teste de Cointegração de Johansen

```

library(urca)

mydba <- merge(datam$QQQ, datam$SPX, datam$DJI) #

# Transformar para log
log_mydba <- log(mydba)

# Teste de Johansen
johansen_test1 <- ca.jo(log_mydba, type = "trace", spec = "longrun", ecdet = "trend", K = 2)
summary(johansen_test1)

##
## #####
## # Johansen-Procedure #
## #####
##
## Test type: trace statistic , with linear trend in cointegration
##
## Eigenvalues (lambda):
## [1] 1.014119e-01 4.463807e-02 2.150309e-02 -5.204170e-17
##
## Values of teststatistic and critical values of test:
##
##          test 10pct  5pct  1pct
## r <= 2 |   6.61 10.49 12.25 16.26
## r <= 1 |  20.49 22.76 25.32 30.45
## r = 0  |  53.00 39.06 42.44 48.45
##
## Eigenvectors, normalised to first column:
## (These are the cointegration relations)
##
##          datam.QQQ.l2 datam.SPX.l2 datam.DJI.l2      trend.l2
## datam.QQQ.l2  1.000000000    1.0000000  1.000000000  1.000000000
## datam.SPX.l2 -0.489907470   -44.2440009 -4.482016208 -2.962931534
## datam.DJI.l2 -0.669210750    74.0188357  4.147478169  1.812738244
## trend.l2     -0.002119118   -0.1658814 -0.001866548 -0.001526793
##
## Weights W:
## (This is the loading matrix)
##
##          datam.QQQ.l2 datam.SPX.l2 datam.DJI.l2      trend.l2
## datam.QQQ.d  -0.05431100 -0.002354433  0.006805940  2.690626e-13
## datam.SPX.d   0.02898314 -0.001846813  0.006095590  3.312385e-13
## datam.DJI.d   0.05404783 -0.001802989  0.001179607  4.195427e-13

johansen_test2 <- ca.jo(log_mydba, type = "eigen", spec = "longrun", ecdet = "trend", K = 2)
summary(johansen_test2)

##
## #####
## # Johansen-Procedure #
## #####
##

```

```
## Test type: maximal eigenvalue statistic (lambda max) , with linear trend in cointegration
##
## Eigenvalues (lambda):
## [1] 1.014119e-01 4.463807e-02 2.150309e-02 -5.204170e-17
##
## Values of teststatistic and critical values of test:
##
##          test 10pct  5pct  1pct
## r <= 2 |   6.61 10.49 12.25 16.26
## r <= 1 |  13.88 16.85 18.96 23.65
## r = 0  |  32.51 23.11 25.54 30.34
##
## Eigenvectors, normalised to first column:
## (These are the cointegration relations)
##
##          datam.QQQ.12 datam.SPX.12 datam.DJI.12      trend.12
## datam.QQQ.12  1.000000000    1.0000000    1.000000000    1.000000000
## datam.SPX.12 -0.489907470   -44.2440009  -4.482016208  -2.962931534
## datam.DJI.12 -0.669210750    74.0188357   4.147478169   1.812738244
## trend.12      -0.002119118   -0.1658814  -0.001866548  -0.001526793
##
## Weights W:
## (This is the loading matrix)
##
##          datam.QQQ.12 datam.SPX.12 datam.DJI.12      trend.12
## datam.QQQ.d  -0.05431100 -0.002354433   0.006805940  2.690626e-13
## datam.SPX.d   0.02898314 -0.001846813   0.006095590  3.312385e-13
## datam.DJI.d   0.05404783 -0.001802989   0.001179607  4.195427e-13
```

4. Estimar o VECM

```
vecm <- cajorls(johansen_test1, r = 1 )

# Vetor de cointegração
vecm$beta

##          ect1
## datam.QQQ.12  1.000000000
## datam.SPX.12 -0.489907470
## datam.DJI.12 -0.669210750
## trend.12      -0.002119118

# Vetor de correção de erro
vecm$rlm

##
## Call:
## lm(formula = substitute(form1), data = data.mat)
##
## Coefficients:
##          datam.QQQ.d  datam.SPX.d  datam.DJI.d
## ect1          -0.05431         0.02898         0.05405
## constant       -0.11111         0.06714         0.12089
## datam.QQQ.dl1   0.25888         0.07241         0.11153
## datam.SPX.dl1   0.42885         0.42131         0.06778
## datam.DJI.dl1  -0.51643        -0.34283        -0.06596
```

Exercício: Represente esses resultados na forma matricial do VECM. Escreva as matrizes:

$$\Delta X_t = (\alpha\beta')X_{t-1} - \Gamma_1\Delta X_{t-1} + e_t$$

onde α é a velocidade de ajuste do vetor de correção de erros e β é um vetor de cointegração.

Equação de Cointegração com Tendência

$$ECT_t = QQQ_t - 0.491633705 \cdot SPX_t - 0.663627794 \cdot DJI_t - 0.002137011 \cdot t$$

Equações do Vetor de Correção de Erro (VECM)

$$\Delta QQQ_t = -0.05468 \cdot ECT_{t-1} - 0.10995 + 0.25709 \cdot \Delta QQQ_{t-1} + 0.38500 \cdot \Delta SPX_{t-1} - 0.46333 \cdot \Delta DJI_{t-1}$$

$$\Delta SPX_t = 0.02772 \cdot ECT_{t-1} + 0.06324 + 0.06776 \cdot \Delta QQQ_{t-1} + 0.39232 \cdot \Delta SPX_{t-1} - 0.30374 \cdot \Delta DJI_{t-1}$$

$$\Delta DJI_t = 0.05136 \cdot ECT_{t-1} + 0.11304 + 0.10281 \cdot \Delta QQQ_{t-1} + 0.03957 \cdot \Delta SPX_{t-1} - 0.02349 \cdot \Delta DJI_{t-1}$$

5. Projetar o VECM

```
library(vars)

## Loading required package: MASS
## Loading required package: strucchange
## Loading required package: sandwich
## Loading required package: lmtest
# Vamos representar o VECM na sua forma VAR:
modelo_var_vecm <- vec2var( johansen_test1, r = 1 )

# forecast
forecast = predict( modelo_var_vecm, n.ahead = 12 )

forecast_nivel <- lapply(forecast$fcst, function(x) {
  x[,1:3] <- exp(x[,1:3]) # média, IC inferior e superior
  x
})

# valores previstos
print(forecast_nivel)

## $datam.QQQ
##      fcst      lower      upper      CI
## [1,] 26795.19 24404.66 29419.88 0.09344814
## [2,] 26822.42 23108.39 31133.37 0.14904222
## [3,] 26865.84 22162.27 32567.67 0.19246444
## [4,] 26935.18 21449.36 33824.03 0.22773836
## [5,] 27029.14 20895.72 34962.87 0.25737131
## [6,] 27144.60 20454.01 36023.70 0.28299907
## [7,] 27278.51 20093.55 37032.64 0.30570051
## [8,] 27428.23 19793.89 38007.09 0.32619972
## [9,] 27591.56 19541.02 38958.79 0.34499445
## [10,] 27766.65 19325.00 39895.83 0.36243609
```

```
## [11,] 27951.95 19138.61 40823.85 0.37877939
## [12,] 28146.18 18976.44 41746.91 0.39421366
##
## $datam.SPX
##      fcst      lower      upper      CI
## [1,] 7605.689 7068.512 8183.690 0.0732466
## [2,] 7656.110 6829.902 8582.263 0.1141936
## [3,] 7705.191 6654.285 8922.065 0.1466332
## [4,] 7753.857 6515.507 9227.570 0.1740054
## [5,] 7802.261 6400.853 9510.494 0.1979823
## [6,] 7850.474 6303.170 9777.610 0.2195212
## [7,] 7898.558 6218.074 10033.206 0.2392200
## [8,] 7946.571 6142.708 10280.154 0.2574748
## [9,] 7994.566 6075.120 10520.464 0.2745602
## [10,] 8042.588 6013.916 10755.591 0.2906748
## [11,] 8090.677 5958.066 10986.627 0.3059665
## [12,] 8138.865 5906.785 11214.413 0.3205491
##
## $datam.DJI
##      fcst      lower      upper      CI
## [1,] 51634.64 48130.95 55393.38 0.07026734
## [2,] 52062.93 46814.04 57900.33 0.10627002
## [3,] 52458.05 45868.58 59994.16 0.13423350
## [4,] 52833.19 45116.60 61869.61 0.15788949
## [5,] 53192.91 44483.86 63607.01 0.17879866
## [6,] 53540.01 43932.93 65247.93 0.19776502
## [7,] 53876.71 43442.39 66817.23 0.21526267
## [8,] 54204.83 42998.82 68331.25 0.23159722
## [9,] 54525.93 42593.33 69801.46 0.24697860
## [10,] 54841.31 42219.64 71236.26 0.26155818
## [11,] 55152.10 41873.17 72642.07 0.27544944
## [12,] 55459.23 41550.43 74023.93 0.28874025
```

Graficamente:

```
library(ggplot2)

# Função para criar gráfico para cada variável
plot_var_forecast <- function(var_name) {

  # Série histórica
  hist_series <- tail(datam[, var_name], 36)

  # Índice para projeção
  start_forecast <- end(hist_series) + 1/12
  forecast_index <- seq(start_forecast, by = 1/12, length.out = 12)

  # Nome no forecast$fcst
  fcst_name <- paste0("datam.", var_name)

  # Valores previstos e intervalos
  pred <- zoo(forecast_nivel[[fcst_name]][, "fcst"], forecast_index)
  lower <- zoo(forecast_nivel[[fcst_name]][, "lower"], forecast_index)
  upper <- zoo(forecast_nivel[[fcst_name]][, "upper"], forecast_index)
```

```

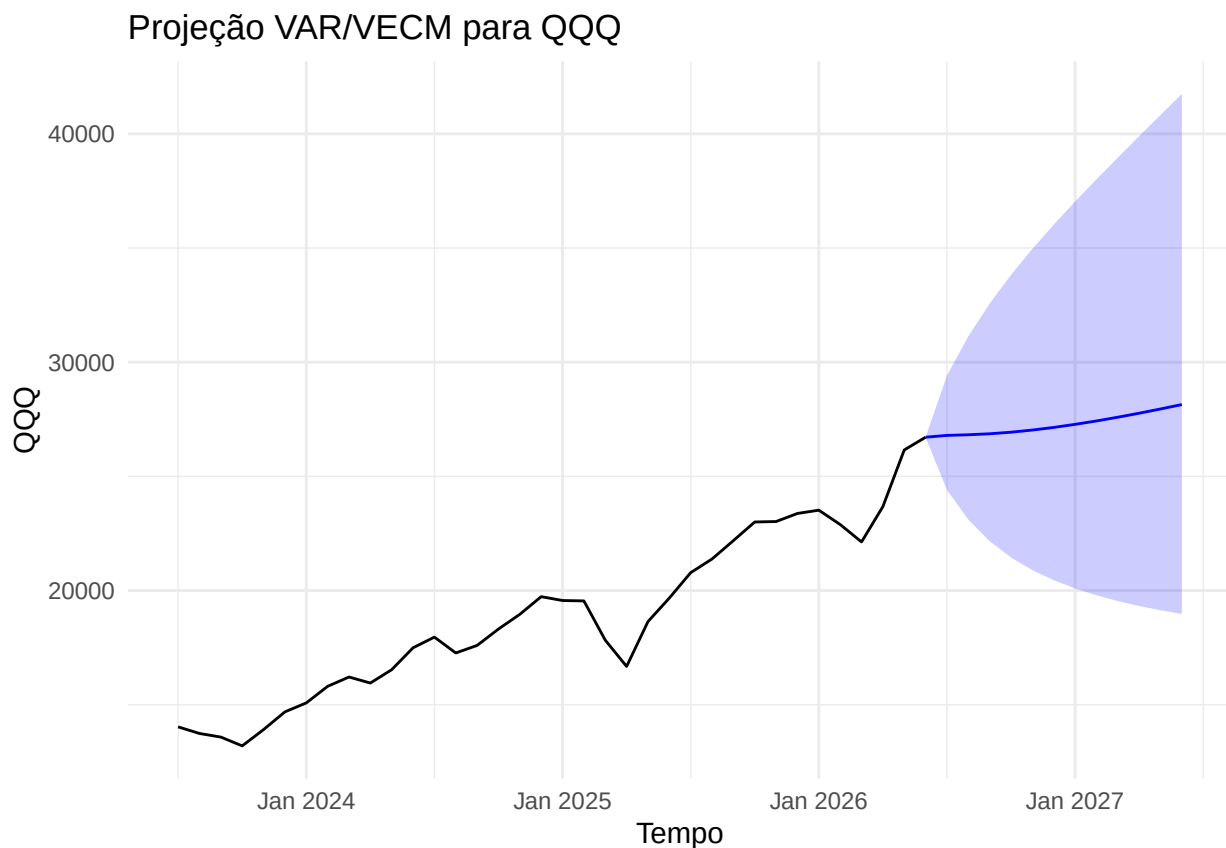
# Adiciona o último ponto da série histórica como primeiro da previsão
pred <- zoo(c(coredata(tail(hist_series, 1)), coredata(pred)), c(end(hist_series), index(pred)))
lower <- zoo(c(coredata(tail(hist_series, 1)), coredata(lower)), c(end(hist_series), index(lower)))
upper <- zoo(c(coredata(tail(hist_series, 1)), coredata(upper)), c(end(hist_series), index(upper)))

# Combinar tudo
combined <- merge(hist_series, pred, lower, upper)
df <- fortify.zoo(combined)
colnames(df) <- c("Date", "Hist", "Forecast", "Lower", "Upper")

# Gráfico
ggplot(df, aes(x = Date)) +
  geom_line(aes(y = Hist), color = "black", na.rm = TRUE) +
  geom_line(aes(y = Forecast), color = "blue", na.rm = TRUE) +
  geom_ribbon(aes(ymin = Lower, ymax = Upper), fill = "blue", alpha = 0.2, na.rm = TRUE) +
  labs(title = paste("Projeção VAR/VECM para", var_name),
       y = var_name, x = "Tempo") +
  theme_minimal()
}

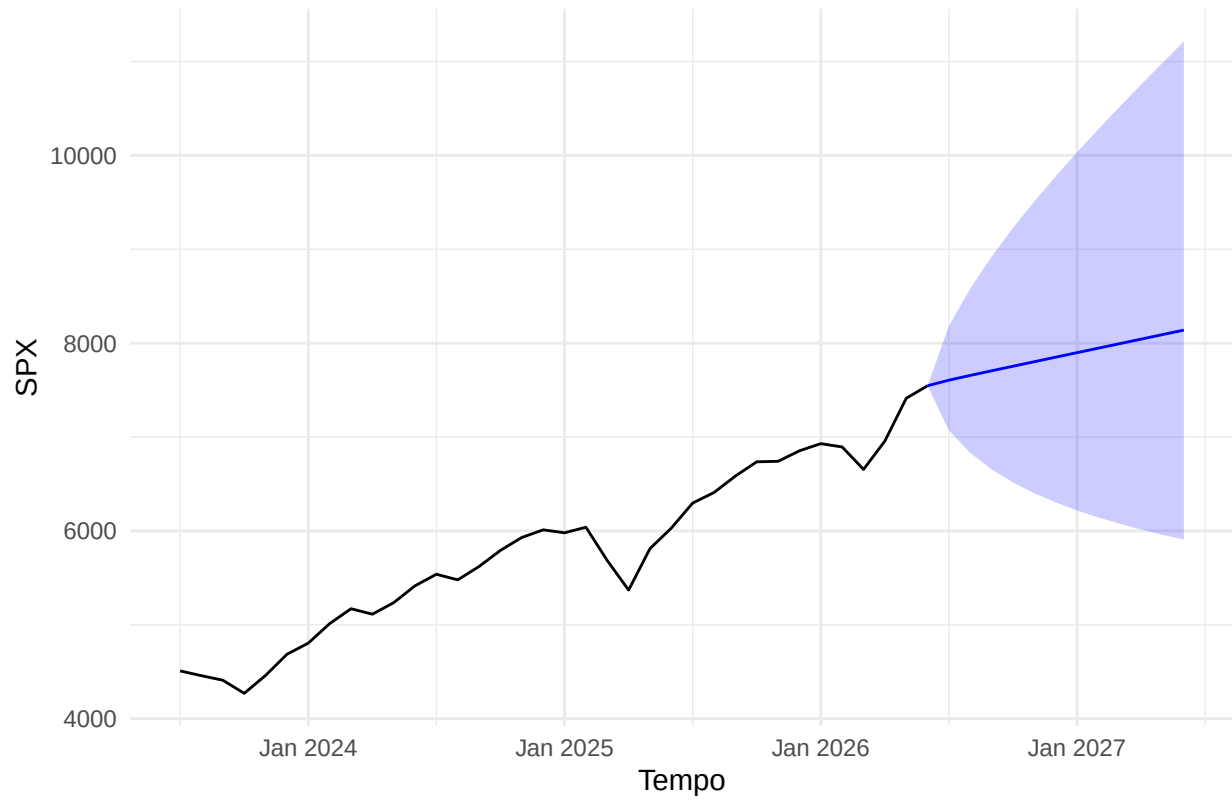
# Gerar gráficos para as 3 variáveis
plot_var_forecast("QQQ")

```



```
plot_var_forecast("SPX")
```

Projeção VAR/VECM para SPX



```
plot_var_forecast("DJI")
```

Projeção VAR/VECM para DJI

